



EHHO-EL: a hybrid method for software defect detection in software product lines using extended Harris hawks optimization and ensemble learning

Mehdi Habibzadeh-khameneh^{1,2} · Akbar Nabiollahi-Najafabadi^{1,2} ·
Reza Tavoli³ · Hamid Rastegari^{1,2}

Accepted: 25 January 2025

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract

Software Product Lines (SPLs) aim to reduce development costs and timelines, while improving quality, but their inherent complexity often results in defects and delays. One of the key challenges in SPLs is selecting relevant features from a vast pool of software metrics for defect detection. To address this issue, this study introduces a hybrid approach combining Harris Hawks Optimization (HHO) with stacking-based ensemble learning. The HHO algorithm is further enhanced with the Chaos Optimization Algorithm (COA) to overcome local optima, making it effective for high-dimensional and complex problems. The method is evaluated using the LVAT and NASA repositories, with four datasets from each source. It achieves accuracies of 92.7%, 91.1%, 96.3%, and 98.4% on LTS1, LTM2, LTL3, and LTV4, and 97.91%, 99.01%, 94.21%, and 90.93% on CM1, JM1, KC1, and PC1. The results highlight the proposed method's enhanced efficiency and faster convergence compared to existing approaches.

Keywords Software product lines · Metaheuristic algorithms · Software defect detection · Harris hawks optimization · Stacking-based ensemble learning · Product defects

1 Introduction

The increasing demand for large-scale software product lines (SPLs) necessitates meticulous attention to feature modeling. Leading commercial organizations have invested significantly in analyzing software defects, while researchers globally continue to explore more accurate and high-quality approaches in this field. Although various methods have been proposed, employing ensemble learning for defect

Extended author information available on the last page of the article

Published online: 03 March 2025

Springer

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

detection and defect-free modeling remains a relatively novel and underexplored approach.

Ensemble learning is a technique that employs multiple models simultaneously to address a problem. Software development is inherently complex, encompassing tasks such as feature selection and software defect resolution, both of which critically influence the quality and efficiency of SPLs. Optimal feature selection focuses on identifying a software product's most essential features, while addressing conflicts often arising from managing competing project requirements or constraints. Industrial experience highlights the significant value of methods and tools that support feature model analysis, making them indispensable in practice [1].

The proposed approach seeks to improve software quality and reliability, boost user satisfaction and acceptance, improve efficiency and productivity in software development, provide greater flexibility and adaptability to evolving user needs, and reduce development costs and time to market. Accurate feature selection is pivotal in achieving higher software quality, while minimizing costs. Moreover, resolving conflicts in feature models is crucial for addressing software defects in SPLs [2]. Several challenges in SPLs include delivering software that optimizes costs, reduces production time, and maximizes quality. High dimensionality and irrelevant features present significant obstacles, complicating the optimal selection of model features necessary to improve efficiency. Ensuring appropriate product configurations that align with customer needs, while adhering to feature model constraints is critical. Failure to comply with these constraints often leads to incompatible product configurations, increasing complexity and defects. Addressing these challenges through the effective defect and defect-free modeling of software products is essential for improving SPL outcomes.

Given these challenges, identifying effective solutions is critical. Metaheuristic algorithms, widely recognized for addressing optimization problems, are typically categorized into two main types: single-solution-based and population-based metaheuristics. This study selects the Harris Hawks Optimization (HHO) algorithm for enhancement due to its proven capability to navigate large search spaces and tackle complex problems. The HHO algorithm demonstrates better performance and speed compared to other optimization methods, making it an ideal choice for this research [3, 4]. This research is divided into two primary phases. The first phase focuses on the optimal selection of product features and the resolution of product configuration defects using an improved HHO algorithm. This improvement increases the accuracy and speed of software defect detection. The proposed method will be compared with existing approaches to validate its effectiveness. The second phase addresses defect and defect-free detection in final products through a two-layer stacking-based ensemble learning method. Machine learning techniques are pivotal in developing predictive models from available data, significantly supporting this process [5]. The experiments will utilize datasets from the LVAT and NASA repositories to achieve this.

The contributions of this article include:

- Development of a novel metaheuristic method: Based on the HHO algorithm, this method is designed to select optimal features in SPLs.

- Introduction of a combined classifier: A novel approach to building a model for defect detection in SPLs.
- Evaluation of the metaheuristic method: The proposed method is tested on the LVAT and NASA datasets, assessing performance metrics such as accuracy, precision, recall, F1 score, classification error (E), and fitness function.
- Testing of the combined classifier: The classifier is also applied to the LVAT and NASA datasets, with the same evaluation criteria used to assess its effectiveness.
- Comparison with existing methods: The proposed method is systematically compared and evaluated against previous approaches to validate its improvements.

This paper is structured as follows: Sect. 2 reviews related works and relevant literature. Sections 3 and 4 detail the proposed method, including an overview of the Enhanced Harris Hawks Optimization (EHHO) algorithm, an explanation of the HHO phases, the novel fitness function introduced in this study, and the ensemble learning approach employed. These sections also present the experiments, results, and evaluations. Finally, Sect. 5 concludes the paper and offers recommendations for future research.

2 Related works

Over the years, various strategies have been proposed to identify defects in SPLs. However, no research has utilized the combination of metaheuristic algorithms with machine learning to address these defects. Existing solutions to this problem can be categorized into two main groups: the first group comprises research that solely employs metaheuristic algorithms. In contrast, the second group focuses on applying machine learning algorithms in this domain. In the following section, we provide a brief overview and summary of research focusing on metaheuristic algorithms and their different types in Table 1. Subsequently, we examine research solely based on machine learning methods and summarize it in Table 2.

2.1 Feature selection with metaheuristic algorithms

This section highlights research utilizing metaheuristic algorithms. Nature-inspired metaheuristic algorithms are effective tools for feature selection and error identification in decision-making by data owners, thereby improving the quality and performance of software products [6–8]. For instance, Ref. [9] introduces a bidirectional ant colony optimization feature selection (BDACOFS) algorithm inspired by ACOFS. Additionally, a chaotic whale optimization algorithm (CWOA) integrates chaotic search into WOA iterations to address optimization challenges [10]. Furthermore, the BBO algorithm has been applied to optimize feature selection in software product lines (SPL), showcasing its effectiveness [11]. Reference [12] proposes a novel hybrid algorithm which combines IBEA with the accuracy of a mixed-integer linear programming (MILP) reparation. A hybrid version, a hybrid simplified PSO-based feature selection algorithm with the elite strategy (HECSPSO), integrates elite

Table 1 Summary of the research on feature selection using metaheuristic algorithms

Refs.	Year	Research approach	Dataset	Advantages	Disadvantages
[9]	2015	Feature selection using bidirectional ANT optimization	Five datasets from UCI (Iris, Glass, Tae, Vehicle, Wine)	Reduce the number of features, improve classification accuracy, reduce duplicate features	High computational complexity, requiring adjustment of numerous parameters
[10]	2018	Feature optimization using a new algorithm inspired by nature	UCI datasets (ten different datasets including WDBC, MPED, PDD, Cardiocography, Hepatitis, Lung Cancer, SPECTF Heart, Thoracic Surgery, Statlog Heart, ILP)	Improving the performance of the WOA algorithm, reducing computational time, increasing accuracy in feature selection	The need to adjust various parameters, the complexity of implementing the algorithm
[11]	2020	Feature Selection Optimization for HealthCare Software Product Line using BBO Algorithm	A case study of a healthcare application that includes multiple feature models	BBO Algorithm provides the best feature subsets for software product lines	Requires more parameter settings
[12]	2021	Hybrid MILP-IBEA algorithm	Feature models from the LVAT repository (e.g., Linux, uClinux, Fiasco, FreeBSD, eCos)	Improved performance	Requires more parameter settings
[13]	2022	Simplified Particle Swarm Optimization (PSO) with maximum separation and minimum redundancy	UCI Machine Learning Repository	Efficient global search, reduced computational complexity, improved convergence speed	Susceptible to local optima, high computational cost
[14]	2022	Hybrid cosine-sine algorithm and Modified Whale Optimization Algorithm (SCMWOA)	Nineteen datasets from the UCI Machine Learning Repository	Increased accuracy and performance	Complex implementation and resource-intensive computation
[22]	2023	Binary Oriented Feature Selection Crossover Constraints (BOFS-CTC)	UCI Machine Learning Repository	Efficient removal of invalid feature combinations	Requires extensive product testing
[15]	2024	Hybrid optimization algorithm based on golden jackal and gray wolf (GIO-GWO)	Eight basis functions and ten feature selection problems	Improves optimization performance and classification accuracy	Increased algorithmic complexity

Table 1 (continued)

Refs.	Year	Research approach	Dataset	Advantages	Disadvantages
[16]	2024	Hybrid model with Whale Optimization (WOA), Particle Swarm Optimization (PSO), and Firefly Algorithm (FA) using Deep Q-Learning	Various datasets from different software projects	Improved accuracy, precision, recall, AUC, and feature identification	Requires detailed parameter tuning for optimal performance
[17]	2024	Genetic Algorithms (GA), Gray Wolf Optimization (GWO), and Support Vector Machine (SVM) for anomaly detection	Financial data from companies	Effective use of metaheuristic optimization and data mining for feature selection	Efficiency needs further validation under diverse conditions
[18]	2024	Genetic Algorithms (GA), Particle Swarm Optimization (PSO), and Gray Wolf Optimizer (GWO) for feature selection with machine learning algorithms	NSS-KDD and KDDCup10% datasets	Enhances malicious packet identification accuracy	Constraints in high-dimensional data handling, a potential decline in decision-making accuracy
[19]	2024	NSGA-II genetic algorithm for K-NN classification parameter optimization	PROMISE datasets (various software projects)	Improved software defect prediction accuracy	High parameter-setting and computational complexity
[20]	2024	Novel metaheuristic inspired by <i>Philoponella Prominens</i> behavior (escape, sexual cannibalism, predation) for optimization tasks	CEC2017 and CEC2022 benchmarks, 4 engineering problems, and UAV 3D path planning for power line inspection	Innovative design, superior performance in benchmarks, effective balance of exploration/exploitation, and diverse real-world applications	High computational complexity, limited dataset evaluation, parameter sensitivity, and potential result instability
[21]	2025	Hybrid SFO-WOA-PSO-K-means method for intrusion detection	BOT-IOT, UNSW-NB15	High detection accuracy, low execution time, efficient feature selection, robust classifier	Potential high complexity, limited scalability to other types of datasets

Table 2 Summary of the research completed in the field of feature selection using machine learning

Refs.	Year	Research approach	Dataset	Advantages	Disadvantages
[25]	2021	Combined feature selection and machine learning techniques, including boosting, bagging, and stacking, for code smell detection	Datasets from various Java systems in the Qualitas Corpus	Significant improvements in accuracy, AUC, and F-measure	Implementation complexity and increased computational resource requirements
[26]	2021	Ensemble learning techniques (Bootstrap Aggregation) for software defect prediction	Software project datasets (e.g., Jedit4.0, Ant1.7, Camel1.4 from the PROMISE repository)	Reduces variance, prevents overfitting, and improves model stability and accuracy	Requires more computational resources than single-learning methods
[30]	2021	Use of XGBoost and stacking ensemble methods to address data imbalance and high dimensionality in defect prediction	Datasets containing software defect data	Resolves data imbalance and high dimensionality, enhancing prediction accuracy	Increased model complexity and computational cost
[27]	2022	Feature extraction for SPLs using Latent Semantic Indexing (LSI), Principal Component Analysis (PCA), and TF-IDF techniques	Various product versions from three SPLs	Utilizes semantic information for feature identification	Dependency on accurate semantic relationship detection
[31]	2023	Feature selection methods (filtering, packing, embedded approaches)	Diverse datasets from various fields	Improves machine learning model accuracy and reduces computational complexity	Risk of overfitting and challenges in achieving optimal feature selection
[28]	2024	Software defect prediction model using RF, SVM, Naïve Bayes, and ANN	Historical defect datasets from the NASA MDP repository (e.g., CMI, JMI, MC2, MW1, PC1, PC3, PC4)	Combines multiple classifiers to improve prediction accuracy	Requires precise parameter tuning for each classifier
[29]	2024	GOA, GA, and RF for IDS	UNSW-NB15, CIC-DDoS2019, and CIC Bell DNS EXF 2021 datasets	High performance in multiclass classification, excellent results for individual classes	High model complexity

strategy and chaos mechanisms for improved feature selection [13]. The proposed algorithm outperforms SATIBEA across various measures.

In another study, Ref. [14] presents a hybrid cosine-sinusoidal modified whale optimization algorithm (SCMWOA), demonstrating its effectiveness in feature selection and solving engineering problems. Its performance is validated through experiments on the UCI dataset and related case studies. Reference [15] introduces the Golden Jackal-Grey Wolf Hybrid Optimization Algorithm (GJO-GWO), significantly improving classification accuracy and reducing execution time. Additionally, a feature selection method based on the Gray Wolf-Golden Jackal hybrid optimization algorithm further improves classification accuracy and execution time [15].

The hybrid technique, coupled with random forest classification, achieves the highest accuracy of up to 95.41%. Furthermore, Ref. [16] proposes an innovative feature selection approach that integrates the Whale Optimization Algorithm (WOA), Particle Swarm Optimization (PSO), and Firefly Algorithm (FA) with Deep Q-learning. This model achieves up to 9.5% higher precision, 8.5% greater accuracy, 8.3% improved recall, 4.9% higher AUC, and 5.9% more specific features than existing methods. Similarly, Ref. [17] combines metaheuristic algorithms with machine learning techniques, including Genetic Algorithm (GA), PSO, and Gray Wolf Optimizer (GWO), alongside machine learning algorithms such as Decision Tree (DT), Random Forest (RF), Gaussian Naïve Bayes (GNB), and Logistic Regression (LR) for feature selection from computer network traffic data. The study identifies the combination of PSO and RF in the fitness function as the most effective, achieving an accuracy of 99.57%.

Ref. [18] also presents a method to improve anomaly detection in financial data by employing metaheuristic optimization algorithms such as GA and GWO in conjunction with data mining techniques like Support Vector Machine (SVM).

The highest achieved accuracy with the GWO algorithm is 75.83%. Ref. [19] introduces a novel method to improve defect prediction accuracy in software modules by combining the Non-Dominated Sorting Genetic Algorithm (NSGA-II) with the k-nearest Neighbor (k-NN) algorithm. This approach achieves higher accuracy based on the Matthews Correlation Coefficient (MCC) and demonstrates improved balanced accuracy compared to previous methods, as validated by the dataset. Reference [20] introduces the Philoponella Prominens Optimizer (PPO), a new metaheuristic inspired by the mating and survival behaviors of the spider Philoponella Prominens, including escape after mating, sexual cannibalism, and predation. Reference [21] presents a novel hybrid method based on SFO-WOA and PSO-K-means algorithms for intrusion detection and feature selection optimization, demonstrating high accuracy. For a detailed comparison of various problem-solving approaches for feature selection using metaheuristic algorithms, refer to Table 1.

3 Research related to machine learning algorithms

This section explores research employing machine learning algorithms, widely regarded as innovative tools in software product development. Researchers often combine multiple methods to create effective defect prediction models, each

contributing to identifying and predicting software components prone to errors, thereby improving the quality of the final product [23, 24].

For instance, Ref. [25] combines feature selection with machine learning techniques to improve code smell detection. This approach improves accuracy by boosting, bagging, and stacking methods, with gains of up to 21.43%. Furthermore, the Area Under the Curve (AUC) improves by up to 53.24%, and the F-measure shows enhancements of up to 76.06% by including feature selection. Similarly, reference [26] introduces a method based on Bootstrap Aggregation (Bagging) for predicting software defects. Comparative analysis reveals that this approach achieves accuracies exceeding 90% across various datasets.

In Ref. [27], Latent Semantic Indexing (LSI), Principal Component Analysis (PCA), and TF-IDF techniques are compared for feature extraction from source code. The results show that LSI outperforms the other methods, achieving a precision of 68.6% and a recall of 87.7%. Ref [28] proposes a software defect prediction model using an intelligent ensemble learning approach that integrates RF, Support Vector Machine (SVM), Naïve Bayes, and Artificial Neural Networks (ANN). The VESDP model demonstrates improved accuracy compared to other techniques.

Reference [29] also introduces a hybrid feature selection method to enhance Intrusion Detection Systems (IDS) in cloud environments. This method combines the Grasshopper Optimization Algorithm (GOA) and Genetic Algorithm (GA) with RF. The method achieves evaluation accuracies of 98%, 99%, and 92% on the UNSW-NB15, CIC-DDoS2019, and CIC Bell DNS EXF 2021 datasets.

Table 2 provides a detailed comparison of various problem-solving approaches for optimal feature selection using machine learning algorithms.

Based on the extensive research and models reviewed, further investigation into feature extraction techniques and advanced classification methods for identifying software defects within SPLs is essential. This study builds upon existing work by optimizing metaheuristic algorithms and enhancing classifier performance. A two-step approach is employed to evaluate the proposed model's effectiveness. The HHO algorithm is utilized in the first step for optimal feature selection. In the second step, ensemble learning techniques are applied for software defect detection in SPLs. The results demonstrate high accuracy across the evaluated datasets, confirming the effectiveness of the proposed approach.

4 Proposed method

The SPL comprises a set of applications built on a common architecture and shared components, each incorporating unique features. Variations in these features arise from differing operating environments and user requirements within the production line. Software defects occur when the selection of configuration features violates configuration rules or constraints, resulting in inconsistent products. Key challenges in this domain include high feature dimensionality, the presence of irrelevant features, and the identification of software defects.

Metaheuristic algorithms offer a solution by reducing the dimensionality of feature sets, thereby aiding in the detection and resolution of software defects.

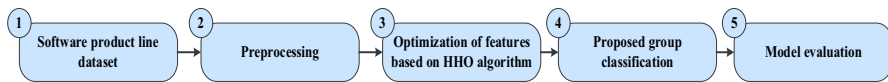


Fig. 1 General steps of the proposed research process

Table 3 Comparative Analysis of Metaheuristic Algorithms

Algorithm	Convergence speed	Optimization accuracy (%)	Local optima avoidance	Computational cost
HHO	High	95.6	Excellent	Medium
PSO	Medium	88.2	Good	Low
GA	Low	82.1	Medium	Low
DE	Medium	89.7	Medium	Medium

This research optimizes the HHO algorithm to improve feature selection and address defects in SPLs. The proposed approach minimizes production time and costs, while improving product efficiency and quality. Rapid defect prediction through an intelligent model is critical for determining whether a product is defect-free. A machine learning-based model is developed to achieve this, employing ensemble learning techniques to detect defects and identify defect-free products efficiently.

A feature set is initially selected using the population-based HHO algorithm [32], which guides the objective function toward optimal values. However, despite its effectiveness in identifying optimal solutions for specific problems, HHO can sometimes stagnate in local optima. The general procedure of the proposed method is shown in Fig. 1.

This research proposes an improved HHO algorithm to address optimization problems. The proposed improvement begins by increasing the initial population's diversity by generating more varied and random solutions. This enhancement broadens the search space exploration, mitigating the risk of the algorithm becoming trapped in local optima during the early stages. Various chaotic maps are employed to achieve this diversity. The chaotic sequences in the Chaos Optimization Algorithm (COA) further guide the HHO search process toward the global optimum in a more structured manner, improving convergence and reducing the likelihood of premature convergence. A comparative analysis of HHO with over 20 metaheuristic algorithms, including PSO, GA, and Differential Evolution (DE), highlights its superior performance in terms of convergence speed and optimization accuracy. A detailed comparison is shown in Table 3.

The key features of the products are represented as a feature vector, with each vector treated as a member of the HHO population. The algorithm selects the vector containing the most significant features, optimizing the objective function. In the second phase, stacked ensemble learning models and classifies SPL products, facilitating efficient defect detection and recognition.

4.1 Preprocessing

Feature selection approaches are fundamental to data mining, machine learning, and knowledge extraction. These methods often require data preprocessing to uncover patterns and achieve optimal classification. Key preprocessing steps include cleaning data, noise removal, normalization [33], and eliminating inappropriate data.

Noise removal was performed by identifying and correcting outliers using statistical methods, such as Z-scores. Normalization, a critical step in data mining preprocessing, ensures that feature values are scaled within a specific range, thereby improving the consistency and comparability of data. This study applied min–max scaling to standardize the range of feature values. This transformation improves the efficiency of knowledge extraction processes by ensuring standardized feature representation in software products.

4.2 Optimization of features based on the improved HHO

In feature selection, it is essential to eliminate irrelevant and redundant features to identify the most effective features in both large and small datasets. Including unnecessary features can degrade algorithm performance and increase the likelihood of defects in the final product.

HHO is a swarm intelligence (SI) algorithm proposed by Heidari et al. in 2019 [34], inspired by the hunting behavior of Harris hawks. Traditional optimization methods are generally suitable for simpler and more structured problems, often assuming a relatively straightforward path to the optimal solution. In contrast, advanced optimization algorithms like HHO handle greater complexity, requiring fine-tuning parameters such as population size, iteration count, and other hyperparameters. These algorithms typically produce approximate solutions, making them particularly valuable for problems with nonlinear, non-continuous objective functions or those involving multiple local optima. The proposed HHO-based method incorporates two distinct phases: exploration and exploitation. The algorithm transitions from exploration to exploitation by adopting strategies modeled on prey escape behaviors, ensuring a dynamic search for the optimal solution. This process enhances the efficiency and accuracy of feature selection and defect detection. Figures 2 and 3 illustrate the feature selection optimization process and the defect detection framework.

Prey instinctively attempts to escape from threatening situations. In the HHO algorithm [34], the prey's chance of successful escape is modeled by a probability r , where $r < 0.5$ indicates a successful escape and $r \geq 0.5$ represents escape failure before the surprise jump. The energy level $|E| \geq 1$ corresponds to the hawks' search across various areas, initiating the exploration phase of the algorithm. Conversely, when $|E| \leq 1$, the



Fig. 2 The feature optimization process based on the improved HHO

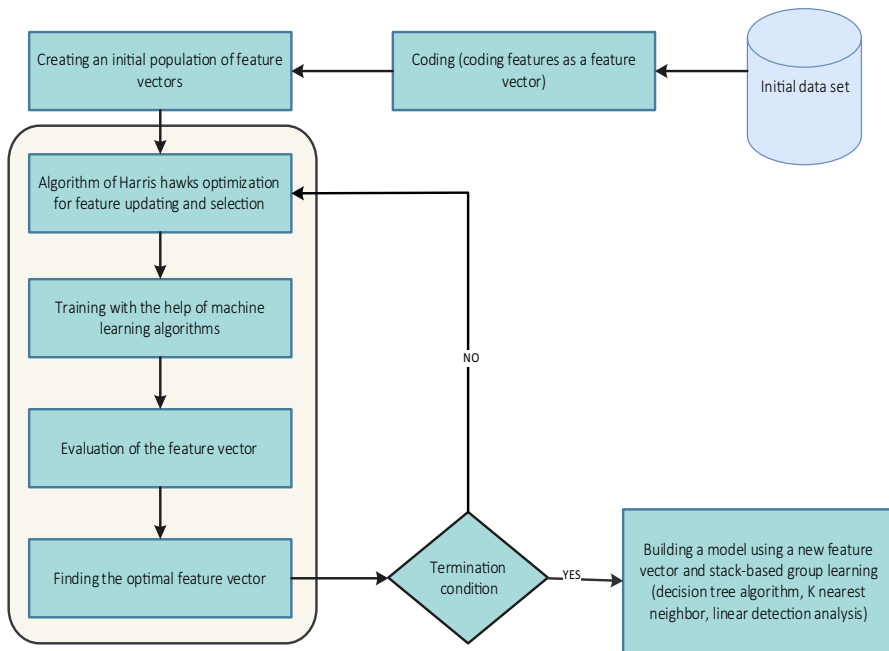


Fig. 3 The conceptual framework of the proposed method for feature selection in SPLs

algorithm transitions to exploitation, using neighboring solutions to refine the search (discovery phase). In summary, $|E| \geq 1$ triggers exploration, while $E < 1$ initiates exploitation. The primary goal of this phase is to improve the diversity of the initial population. The initialization of location plays a critical role in determining population diversity and algorithmic stability. While the HHO algorithm ensures randomness in initial population positions, randomness does not necessarily guarantee uniformity. Chaos Theory generates uniformly distributed random numbers between 0 and 1, and different chaotic maps can be used to create diverse initial populations. Experimenting with various maps and their characteristics can refine randomness to improve performance, particularly by adjusting the initial hawk positions. This approach addresses issues of early convergence and entrapment in local optima by enhancing the application of Chaos Theory during initialization. Figure 4 presents the pseudocode for the HHO algorithm.

Production of the Initial Population of Harris Hawks.

1. Initialize the population of Harris hawks (Using a Chaotic Algorithm instead of a random population).
2. Continue until the termination condition is met:
 1. Calculate the fitness of each hawk.
 2. Set the location of the best-performing hawk as the location of the rabbit (prey).
 3. For each Harris hawk, perform the following:

Algorithm Pseudo-code of HHO algorithm

```

Inputs: The population size N and maximum number of iterations T
Outputs: The location of rabbit and its fitness value Initialize the random population
X, (i = 1, 2, ..., N)
(Using Chaotic Algorithm instead random population)
while (stopping condition is not met) do
  Calculate the fitness values of hawks
  Set Xrabbit as the location of rabbit (best location)
  for (each hawk (X,)) do
    Update the initial energy E0 and jump strength J ▷ E=2rand()-1, J=2(1-rand())
    Update the E using Eq. (1,2)
    if (|E|≥1) then ▷ Exploration phase
      Update the location vector using Eq. (4)
    if (|E|< 1) then ▷ Exploitation phase
      if (r ≥ 0.5 and |E| ≥ 0.5) then ▷ Soft besiege
        Update the location vector using Eq. (5)
      else if (r ≥ 0.5 and E < 0.5) then ▷ Hard besiege
        Update the location vector using Eq. (6)
      else if (r < 0.5 and |E| ≥ 0.5) then ▷ Soft besiege with progressive rapid dives
        Update the location vector using Eq. (7)
      else if (r < 0.5 and |E| < 0.5) then ▷ Hard besiege with progressive rapid dives
        Update the location vector using Eq. (8)
  Return Xrabbit

```

Fig. 4 Pseudocode of the proposed method

- Determine the initial energy state of the prey

$$E_0 = 2\text{rand}() - 1 \quad (1)$$

- Update its jumping power

$$J = 2(1 - \text{rand}()) \quad (2)$$

- Update the energy parameter using the relation

$$(E = 2E_0 (1 - \frac{t}{T})) \quad (3)$$

- If ($E \geq 1$), update the location vector using the following relationship for the exploration phase by Eq. (4):

$$x(t+1) = \begin{cases} X_{\text{rand}}(t) - r_1 |X_{\text{rand}}(t) - 2r_2 X(t)| & q \geq 0.5 \\ X_{\text{rabbit}}(t) - X_m(t) - r_3(LB + r_4(UB - LB)) & q < 0.5 \end{cases} \quad (4)$$

If ($E < 1$), enter the exploitation phase. This step simulates the surprise attack process of the Harris hawk on the prey.

1. If ($E \geq 0.5$) and ($r \geq 0.5$), perform a soft besiege and update the location vector using Eq. (5):

$$x(t+1) = \Delta X(t) - E |JX_{\text{rabbit}}(t) - X(t)| \quad (5)$$

The prey has enough energy and attempts to evade capture, misleading the hawks with random jumps:

2. If ($E < 0.5$) and ($r \geq 0.5$), perform a hard besiege and update the location vector using Eq. (6):

$$x(t+1) = X_{\text{rabbit}}(t) - E|\Delta X(t)| \quad (6)$$

The prey has lower energy and is unable to escape effectively. The hawk performs a hard besiege, directly targeting the prey's location. The location vector is updated using the following equation.

3. If $(E \geq 0.5)$ and $(r < 0.5)$, perform a soft siege with rapid dives and update the location vector using Eq. (7):

$$X(t+1) = \begin{cases} Y & \text{if } F(Y) < F(X(t)) \\ Z & \text{if } F(Z) < F(X(t)) \end{cases} \quad (7)$$

If the prey has sufficient energy to evade successfully, the soft siege phase before the surprise jump is employed. This approach is more strategic than a direct attack, allowing the hawks to plan their movements carefully.

4. If $(E < 0.5)$ and $(r < 0.5)$, perform a hard siege with rapid dives and update the location vector using Eq. (8):

$$X(t+1) = \begin{cases} Y & \text{if } F(Y) < F(X(t)) \\ Z & \text{if } F(Z) < F(X(t)) \end{cases} \quad (8)$$

In this scenario, the prey's lack of energy necessitates a direct and aggressive approach, enabling the hawks to trap the prey effectively through a combination of dives and calculated movements.

At this stage, the HHO algorithm updates its position through four distinct strategies: soft siege, hard siege, soft siege with rapid dives, and hard siege with rapid dives. This iterative process continues until the predefined number of iterations is completed. Once the iterations conclude, the resulting position vectors are evaluated using the fitness function defined in Eq. (9) to determine the optimal solution. The pseudocode for the HHO algorithm is provided as a reference to detail these steps and strategies.

$$\text{Fitness} = aE + b \frac{|S|}{|F|} \quad (9)$$

$$E = 1 - \left(\frac{\text{Accuracy} + \text{Precision} + \text{Recall}}{3} \right) \quad (10)$$

$$\text{Accuracy} = \frac{\text{Number of Corrected Predicts}}{\text{Total Number of Samples}} \quad (11)$$

$$b = 1 - a \quad (12)$$

In the equations above, the value of a is typically determined experimentally and is generally set above 0.9. The term $|S|$ represents the total number of selected features, while $|F|$ denotes the total number of features in the dataset. The parameter E corresponds to the classification error defined in Eq. (10), while Eq. (11) specifies classification accuracy based on the machine learning classifier. A lower value of the fitness function indicates a better optimization outcome. Ultimately, the rabbit's position, which corresponds to the vector of the best-performing hawk relative to the prey, is returned as the solution. The flowchart of the proposed method is shown in Fig. 5.

During the exploitation phase, the Harris hawk's surprise attack on the prey is simulated. The process begins with inputting features from the LVAT and NASA datasets. These features are then encoded into vectors, where each feature is represented in a binary state: zero indicates that the feature is not selected, while a value of one signifies its selection for the product. This encoding allows each feature vector to function as an array and as a member of the population within the HHO algorithm. An intelligent feature selection process is essential since each feature vector

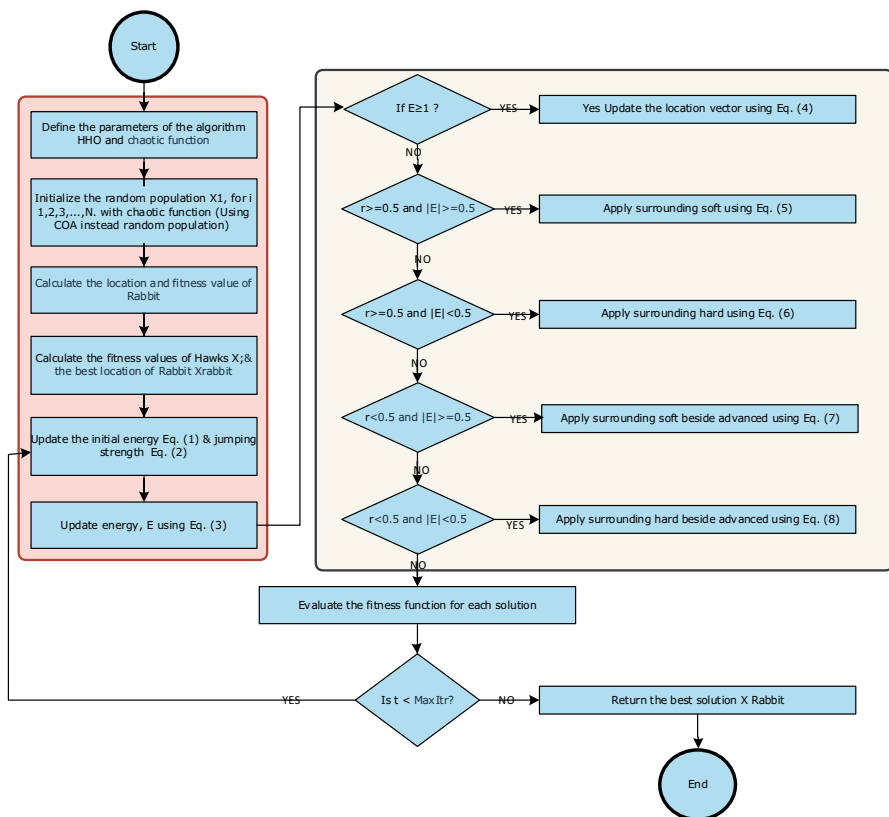


Fig. 5 Flowchart for the proposed feature selection in SPLs

may exhibit various states. This process aims to optimize the selection of features to minimize design costs and enhance efficiency.

The fitness function plays a crucial role in evaluating the selected feature vectors, identifying those that optimize costs and improve the quality of software products within Software Product Lines (SPLs). In the proposed method, successive iterations of the HHO algorithm update the feature vectors or the population of Harris hawks. During each iteration, certain features may be eliminated to refine the solution. The resulting feature vector is then used to construct a model employing machine learning algorithms, facilitating effective defect detection.

While the HHO algorithm demonstrates robust performance across various optimization problems, its susceptibility to becoming trapped in local optima is a significant limitation. To address this, enhancing the diversity of the initial population is critical. This diversity is introduced during the initialization phase, where methods are devised to ensure a varied and well-distributed population. The subsequent sections explain the Harris Hawks methodology and the optimization approach adopted in this study.

We implemented the combined HHO algorithm with the COA to solve the G10 problem from the CEC2006 benchmark, conducting 30 simulation runs. To address the constraints associated with the G10 problem, a penalty method was employed to manage constraint violations effectively. The performance of the proposed method (COA + HHO) was evaluated against four well-established optimization algorithms: the Marine Predator Algorithm (MPA) [35], Original HHO, PSO, and GA. The comparative analysis demonstrates that the COA + HHO approach outperforms these algorithms in terms of both accuracy and convergence speed. Table 4 provides the average objective function values, standard deviation, and key parameters such as population size, number of iterations, mutation rate, and crossover rate.

4.3 Proposed ensemble model

This research proposes a two-layer ensemble classifier for software defect detection, utilizing stack-based ensemble learning. The approach employs diverse machine learning techniques to integrate multiple classifiers, enhancing detection accuracy and reliability.

Table 4 Comparison of Objective Function Values and Constraint Violations for CEC2006 G10

Algorithm	Average objective value	Average constraint violation	Standard deviation	Population size	Number of iterations	Mutation rate	Crossover rate
COA + HHO	0.0051	0.001	0.002	50	1000	0.1	0.9
MPA	0.0192	0.009	0.006	50	1000	0.15	0.85
Original HHO	0.0204	0.010	0.008	50	1000	0.1	0.8
PSO	0.0458	0.020	0.015	50	1000	0.1	0.9
GA	0.0531	0.030	0.018	50	1000	0.05	0.7

4.3.1 Details of the proposed method

Ensemble classifier approaches use multiple simple classifiers to improve predictive accuracy. Initially, these classifiers independently create models, and their outputs are combined using voting, fusion, or learning methods to determine the final class. Prominent ensemble methods include bagging, boosting, and stacking. Among these, stacking stands out by combining the strengths of multiple base models. It uses diverse models to generate predictions, followed by a final model synthesizing these outputs to make the ultimate decision. This layered approach improves accuracy by addressing the limitations of individual models and frequently outperforms single-model approaches.

In complex data scenarios, stacking proves particularly effective by uncovering intricate interactions between models and datasets that base models might overlook. Its flexibility allows the integration of various base models, such as decision trees, K-NN, and discriminant analysis, making it adaptable to the specific requirements of different problems. The method's efficacy lies in aggregating results from diverse models, producing robust solutions with more accurate and comprehensive outcomes. In this study, the ensemble strategy is applied after feature selection is optimized using the improved HHO algorithm, ensuring that only the most relevant features contribute to the classification process.

A two-layer ensemble classifier based on machine learning is introduced in the second phase of the proposed algorithm. The first layer consists of multiple meta-classifiers constructed using three base learning methods: Decision Tree, Discriminant Analysis, and k-nearest neighbor (K-NN). These base models independently generate predictions at the initial level. The second layer integrates the outputs (predicted labels) from the first layer of classifiers into a high-level meta-classifier. At this top level, XGBoost models consolidate the base classifiers' predictions, resulting in a unified and optimized final output. This hierarchical structure enhances the algorithm's predictive accuracy by leveraging the strengths of diverse learning methods, while mitigating individual model weaknesses. Employing XGBoost at the top level enables more robust learning by effectively leveraging the strengths of the base models, while mitigating errors from individual classifiers. This approach ensures the ensemble benefits from XGBoost's better ability to handle complex interactions, feature importance, and outlier robustness. Although the multilayered structure introduces additional computational overhead, the improved accuracy and robustness of predictions typically justify this trade-off, making the method particularly advantageous for complex and high-stakes applications. The proposed method utilizes raw data from the NASA dataset and the LVAT repository, extracting features through machine learning algorithms in the first layer. The main structure of the method, shown in Fig. 4, employs a two-layer architecture with distinct classifiers at each layer. Specifically:

- (1) The input data is given to the second layer using machine learning-based meta-classifiers.
- (2) The first layer outputs are given to a high-level meta-classifier. Leveraging multi-layer learning, the resultant output is expected to exhibit enhanced performance.

Figure 6 illustrates the iterative process of stack-based ensemble learning, highlighting its critical role in enhancing the performance of algorithms susceptible to converging on numerous local optima (Figs. 7, 8, 9, 10).

5 Experiments and results

This section presents the simulations and tests performed on the proposed method. It begins with a description of the dataset, followed by the configuration and optimization of the model parameters. Finally, a comparative analysis compares the proposed method with established and novel approaches.

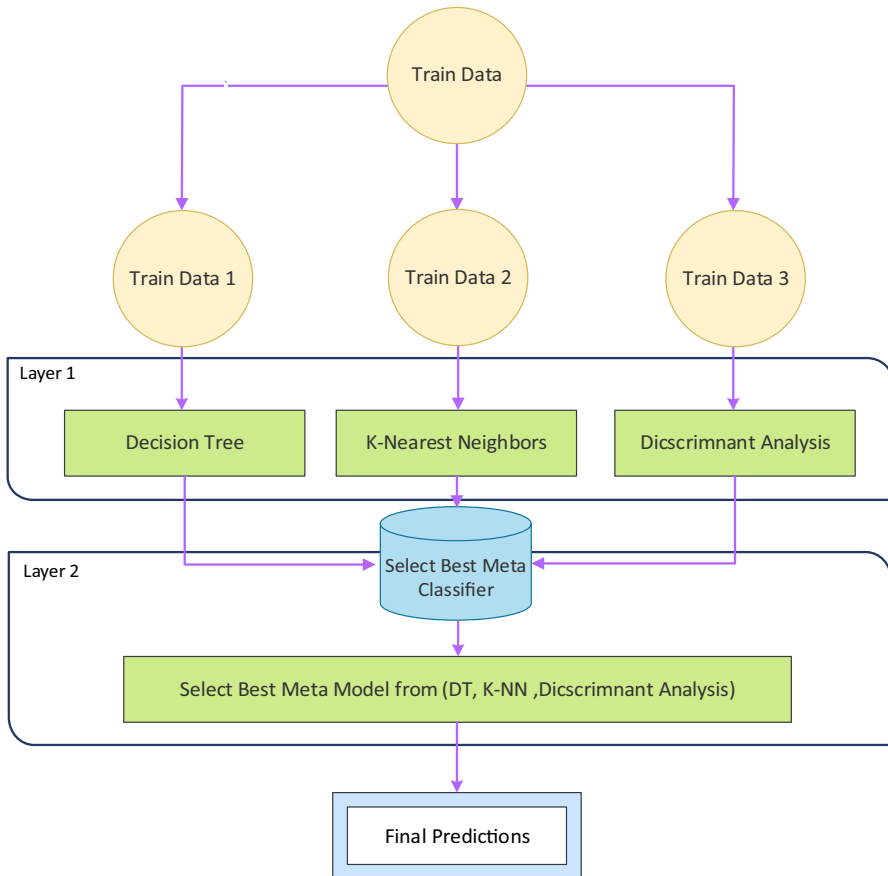


Fig. 6 Proposed software defect detection model with stack-based ensemble learning

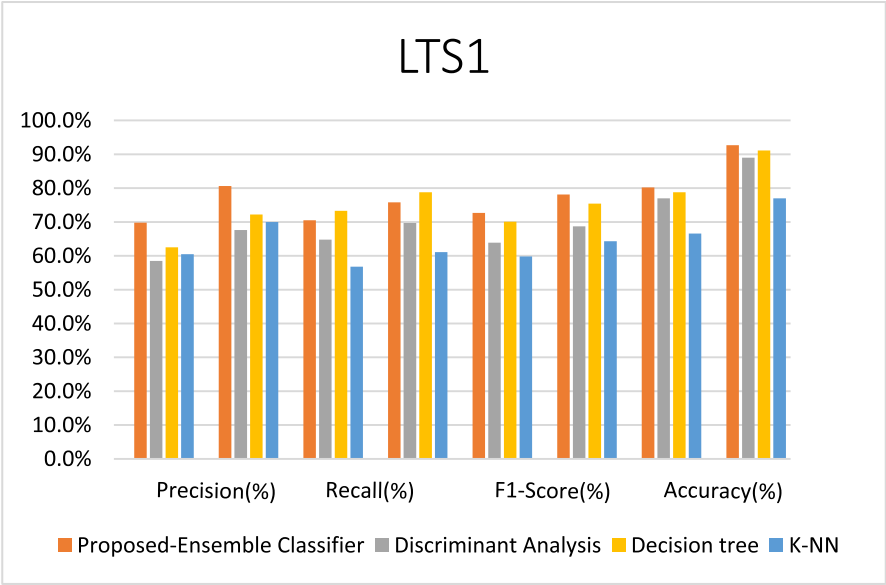


Fig. 7 Evaluation based on Precision, Recall, F1-Score, and Accuracy criteria on the LTS1 dataset

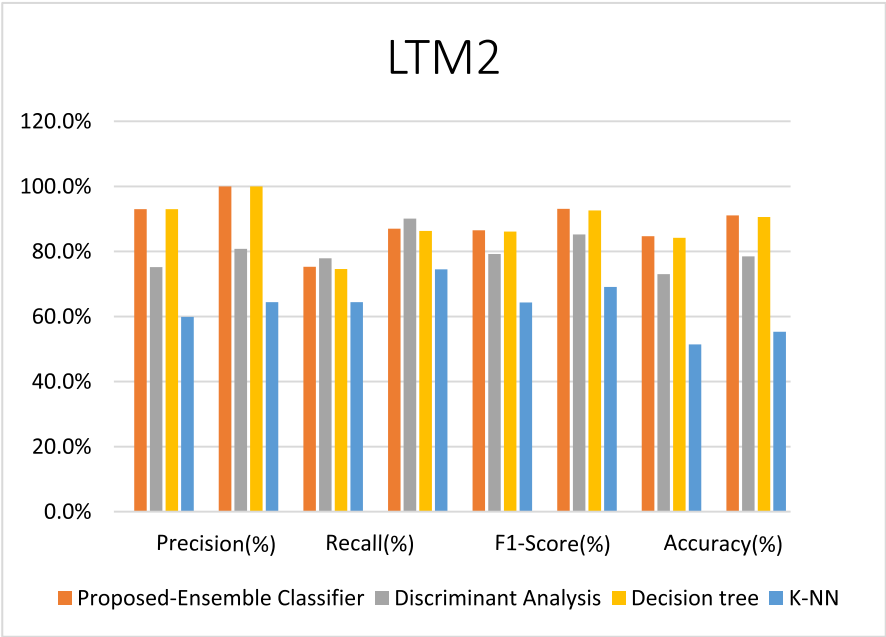


Fig. 8 Evaluation based on Precision, Recall, F1-Score, and Accuracy criteria on the LTM2 dataset

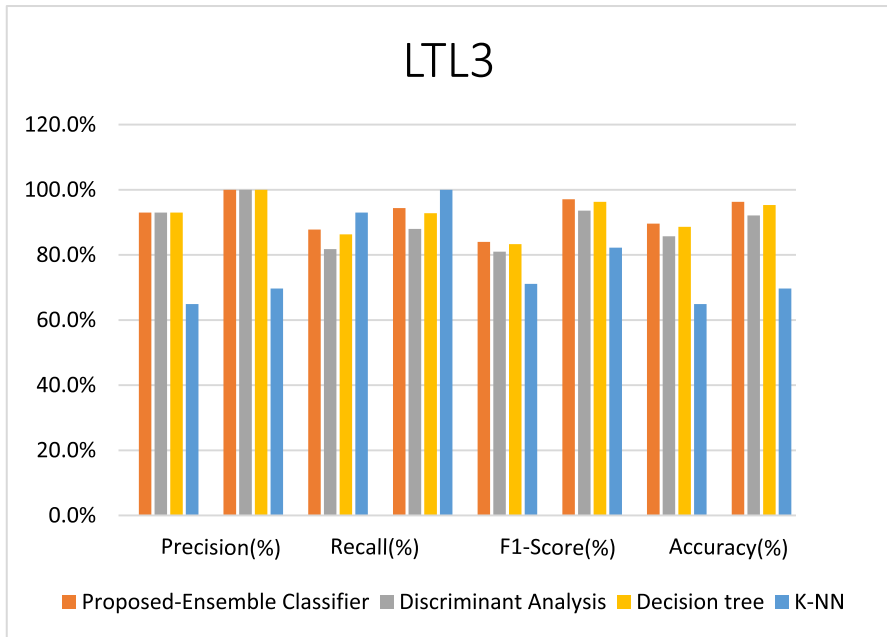


Fig. 9 Evaluation based on Precision, Recall, F1-Score, and Accuracy criteria on the LTL3 dataset

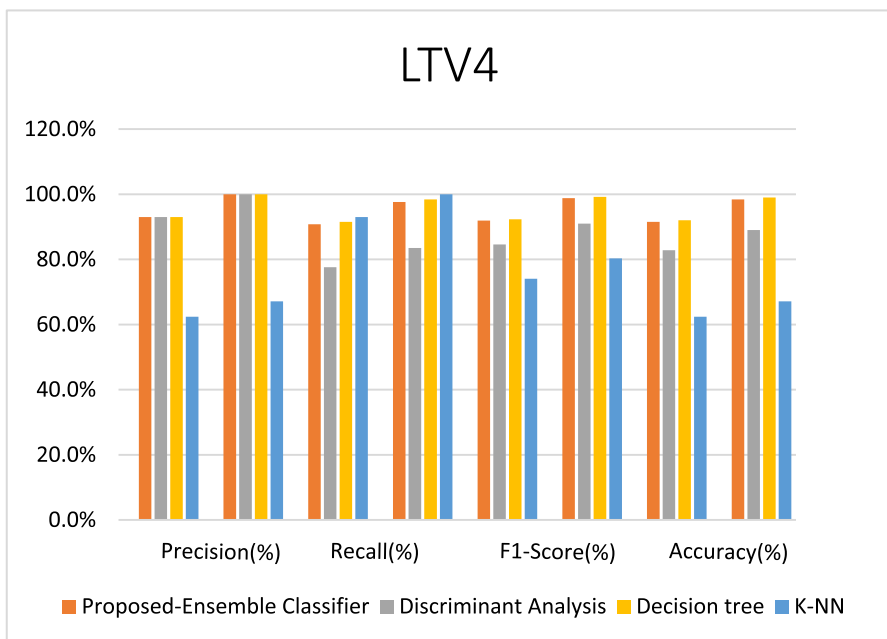


Fig. 10 Evaluation based on Precision, Recall, F1-Score, and Accuracy criteria on the LTV4 dataset

Table 5 Features of the NASA and LVAT datasets

	Dataset	Programming language	Features	Modules	Error percentage	Classes
NASA	Cm1	C	21	498	%10	2
	Jm1	C	21	10,885	%19	2
	Pc1	C	21	1109	%7	2
	Kc1	C + +	21	2109	%15	2
LVAT	LTS1	–	17	479	–	2
	LTM2	–	39	479	–	2
	LTL3	–	52	479	–	2
	LTV4	–	100	479	–	2

5.1 Datasets

The proposed method is developed for software defect detection and evaluation using the LVAT¹ and NASA² datasets, widely recognized benchmark datasets in recent studies. Employing diverse datasets is essential for validating the proposed method's effectiveness, as it enables testing the model's generalizability and robustness across various data types.

To evaluate the efficacy of the proposed approach, the configuration files from the Linux operating system version 2.6.28.6 are utilized, comprising 6888 features and 343,944 constraints [2, 36]. This dataset, available as a public repository within the LVAT, examines numerous variable complexities in the Linux operating system, introducing substantial challenges due to its extensive feature count. The SPLOT tool [37] was employed to generate Feature Models (FMs) of varying sizes (small, medium, large, and very large) to validate the EHHO algorithm. The constraints in these automated FMs follow a normal distribution, as outlined in the accompanying table, to assess the performance of the proposed method.

Additionally, software projects from the NASA Metrics Data Program (MDP) dataset are incorporated into this study. These projects include applications designed for satellite flight control, ground data storage management, and spacecraft instrumentation. Four commonly referenced datasets from this repository are included, each containing software modules with quality-related input metrics. Each module is manually labeled as either defective or non-defective based on the presence or absence of bugs, determined during a testing phase [38, 39]. Table 5 provides further insights into the datasets used in this study. The performance of the proposed method varies across datasets due to factors such as dataset size and complexity, feature distribution, noise and outliers, dimensionality, and dataset-specific patterns.

¹ <https://code.google.com/archive/p/linux-variability-analysis-tools/>.

² https://figshare.com/collections/NASA_MDP_Software_Defects_Data_Sets/4054940/1.

5.2 Evaluation metrics

Formulating the fitness function is critical to metaheuristic algorithms in addressing problem-solving tasks. This research focuses on model construction and classification, highlighting the importance of accuracy, recall, precision, and F1-score metrics in determining an effective encoding strategy. Consequently, it is essential to incorporate all four criteria into the fitness function. Evaluating classification performance begins with constructing the confusion matrix for the test dataset, as shown in Table 6. Following this, the metrics are computed to assess the model's performance comprehensively:

True Negative (TN) refers to the number of records, where the true category is a *defect*, and the classification algorithm correctly identifies them. True Positive (TP) denotes the number of records, where the true category is *No defect*, and the classification algorithm accurately classifies them as *No defect*. Conversely, False negative (FN) represents the count of records, where the true category is *No defect*, but the classification algorithm incorrectly classifies them as a *defect*. Similarly, False positive (FP) is the number of records, where the true category is *defect*, but the classification algorithm mistakenly classifies them as *No defect* [40].

$$\text{Recall}^{\text{defect}} = \frac{\text{TN}}{\text{FP} + \text{TN}} \quad (13)$$

$$\text{Recall}^{\text{no defect}} = \frac{\text{TP}}{\text{FN} + \text{TP}} \quad (14)$$

$$\text{Precision}^{\text{no defect}} = \frac{\text{TN}}{\text{TN} + \text{FN}} \quad (15)$$

$$\text{Precision}^{\text{defect}} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (16)$$

$$\text{F1 - Score} = 2 * \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}} \quad (17)$$

The Recall^x criterion measures the recall of category X relative to all records labeled as X. It is calculated using Eqs. (13) and (14), reflecting the classifier's ability to identify category X across all instances labeled as X. correctly This criterion highlights the proportion of correctly classified instances of X concerning its total occurrences in the dataset. In contrast, the Precision^x criterion evaluates the precision of the classification prediction, as shown in Eqs. (15) and (16). It quantifies the reliability of the

Table 6 Confusion matrix

	Defect	No defect
Defect	TP	FN
No defect	FP	TN

classification output for category X, offering insight into how many predicted instances of X are accurate.

The Accuracy criterion, defined in Eq. (11), provides a holistic measure of classification performance across all categories. Additionally, the F1-score (calculated as the harmonic mean of precision and recall) is expressed in Eq. (17). These metrics collectively evaluate the classifier's performance. The optimal features identified through iterative runs of the HHO algorithm are selected based on Precision, Recall, Accuracy, and F1-score metrics in conjunction with the fitness function (Eq. (9)).

5.3 Parameter setting

This section focuses on optimizing the parameters of the Harris Hawks metaheuristic algorithm and provides a detailed explanation of the stack-based ensemble learning approach.

5.3.1 HHO algorithm parameter setting

To optimize the parameters of the HHO algorithm, we employed a strategy involving careful parameter initialization based on experimental results, combined with the integration of the COA for improved parameter tuning. The model designer must select hyperparameters, which significantly influence model performance, using experience, heuristics, optimization techniques, or automated methods. This process, known as hyperparameter fine-tuning, involves identifying the optimal set of hyperparameters to achieve better performance.

Random Search is a notable method among the various approaches for hyperparameter tuning. Unlike Grid Search, which evaluates all possible combinations within the search space, Random Search selects value combinations at random. This approach is computationally efficient, and, while it may not always identify the absolute best hyperparameters, it often yields models with results close to optimal [41]. Extensive experimental tests were conducted to identify the critical hyperparameters of the HHO algorithm. Table 7 details the values considered during the optimization process.

5.4 Performance evaluation of the proposed model in two states before and after optimization

Our investigation sought to assess the proposed model's effectiveness compared to alternative methods using key performance metrics such as accuracy, precision, recall, and F1 score. The evaluation dataset was obtained from the LVAT repository, and the performance of our approach was compared to that of pre-existing methods. These comparisons are based on four distinct datasets, shown in Table 8 and illustrated through the figures provided in Sect. 4.4.

For the LTS1 dataset, the proposed method achieved the highest precision at 80.6%, outperforming the decision tree (72.2%), K-NN (70%), and linear discriminant analysis (67.6%). Regarding recall, the decision tree recorded the highest

Table 7 Hyper parameter configuration used in the proposed HHO algorithm

Hyperparameters	Values
Population Size (N)	5
Upper search bound (UB)	1
Lower search bound (LB)	0
Number of iterations (T)	100
Number of repeated runs (M)	20
Fitness function constant	0.99

value at 78.8%, with the proposed ensemble approach following at 75.8%, then linear discriminant analysis at 69.7%, and K-NN at 61.1%. The proposed method, leveraging ensemble learning, also demonstrated the highest F1 score, while K-NN exhibited the lowest. Regarding accuracy, the proposed method surpassed all others, followed by linear discriminant analysis, the decision tree, and K-NN.

For the LTM2 dataset, the proposed approach and the decision tree achieved perfect precision at 100%, with linear discriminant analysis following at 80.8% and K-NN at 64.4%. Concerning recall, linear discriminant analysis performed the best at 90.1%, followed by the proposed ensemble approach at 87%, the decision tree at 86.3%, and K-NN at 74.5%. The ensemble learning combination once again demonstrated the highest F1 score, with K-NN recording the lowest. Regarding accuracy, the proposed method led the results, followed closely

Table 8 Comparison of Accuracy, Precision, Recall and F1-score measure of the proposed method with different classifiers and datasets

Method	Data set	Precision (%)		Recall (%)		F1-Score (%)		Accuracy (%)	
		Before	After	Before	After	Before	After	Before	After
Proposed-Ensemble Classifier	LTS1	69.8	80.6	70.5	75.8%	72.7	78.1	80.2	92.7
	LTM2	93.0	100.0	75.3	87.0	86.5	93.1	84.7	91.1
	LTL3	93.0	100.0	87.8	94.4	84.0	97.1	89.6	96.3
	LTV4	93.0	100.0	90.8	97.6	91.9	98.8	91.5	98.4
Discriminant Analysis	LTS1	58.5	67.6	64.8	69.7	63.9	68.7	77.0	89.0
	LTM2	75.2	80.8	77.9	90.1	79.2	85.2	73.0	78.5
	LTL3	93.0	100.0	81.8	88.0	81.0	93.6	85.7	92.1
	LTV4	93.0	100.0	77.6	83.5	84.6	91.0	82.8	89.0
Decision tree	LTS1	62.5	72.2	73.3	78.8	70.1	75.4	78.8	91.1
	LTM2	93.0	100.0	74.6	86.3	86.1	92.6	84.2	90.6
	LTL3	93.0	100.0	86.3	92.8	83.3	96.3	88.6	95.3
	LTV4	93.0	100.0	91.5	98.4	92.3	99.2	92.0	99.0
K-NN	LTS1	60.5	70.0	56.8	61.1	59.8	64.3	66.6	77.0
	LTM2	59.9	64.4	64.4	74.5	64.3	69.1	51.4	55.3
	LTL3	64.9	69.7	93.0	100.0	71.1	82.2	64.9	69.7
	LTV4	62.40	67.10	93.00	100.00	74.07	80.30	62.40	67.10

Table 9 Performance Comparison of Metaheuristic Algorithms on LVAT Dataset

Algorithm	Dataset	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)	Convergence time (ms)
COA + HHO	LVAT	96.3	94.2	97.1	95.6	1100
Original HHO		92.8	90.5	94.0	91.2	1300
PSO		87.5	85.3	88.7	86.5	1600
GA		85.2	82.9	86.5	84.0	1900

by the decision tree, linear discriminant analysis, and K-NN, all of which performed well.

For the LTL3 dataset, 100% precision was achieved by the proposed approach, linear discriminant analysis, and the decision tree, while K-NN followed with a precision of 69.7%. Regarding recall, K-NN led with 100%, followed by the proposed ensemble approach at 94.4%, the decision tree at 92.8%, and linear discriminant analysis at 88%. The proposed method outperformed others regarding the F1 score, with K-NN recording the lowest value. Accuracy results followed a similar pattern, with the proposed method leading, followed by the decision tree, linear discriminant analysis, and K-NN in that order.

For the LTV4 dataset, the highest precision of 100% was achieved by the proposed method, linear discriminant analysis, and the decision tree, with K-NN scoring 67.1%. Regarding recall, K-NN recorded the highest value at 100%, followed by the decision tree at 98.4%, the proposed ensemble approach at 97.6%, and linear discriminant analysis at 83.5%. The decision tree method achieved the highest F1 score, while K-NN recorded the lowest. Regarding accuracy, the decision tree method ranked first, followed by the proposed approach, linear discriminant analysis, and K-NN.

The advantage of the proposed method lies in its integration of advanced techniques, which collectively enhance model performance. The Harris Hawks Optimization (HHO) algorithm facilitates an efficient global search, while avoiding local optima, and the Chaos Optimization Algorithm (COA) enhances exploration, mitigating search limitations. Additionally, Ensemble Learning (Stacking) combines the outputs of multiple models, reducing bias and variance to improve prediction accuracy and robustness to data variations. This synergistic combination enables the proposed model to effectively manage complex, high-dimensional datasets, addressing the shortcomings of individual methods.

To further evaluate the effectiveness of the proposed COA + HHO algorithm, we compared its performance against other metaheuristic algorithms, including the original HHO, PSO, and GA. Tables 9 and 10 summarize the key performance metrics and parameter configurations used in other algorithms.

5.5 Performance analysis of the proposed model

Evaluating model convergence is essential for optimizing performance and often involves fine-tuning hyperparameters. Observing how different settings influence convergence behavior makes determining the optimal hyperparameter values more manageable. Convergence is typically assessed using convergence curves, which depict the progression of the optimization algorithm's objective function over iterations. When additional iterations no longer yield significant improvements in the objective function, the algorithm is considered to have converged. This study presents convergence curves for the fitness function of the HHO algorithm across all four datasets, highlighting the algorithm's ability to converge toward the optimal solution effectively.

As shown in the figures in Sect. 4.5, the HHO algorithm demonstrates a rapid convergence rate, reflecting the effectiveness of the proposed fitness function in minimizing features, while maximizing the feature reduction rate. Additionally, the algorithm's performance during the feature selection stages significantly enhances efficiency, contributing to a high accuracy rate. The algorithm efficiently navigates the solution space in the exploration phase to identify promising regions. It then transitions seamlessly from exploration to exploitation, refining the selected features precisely. This balance between exploration and exploitation enables the HHO algorithm to thoroughly investigate the solution space, ensuring the identification of an optimal solution.

The optimized HHO algorithm demonstrates a consistent and gradual convergence rate across most datasets, effectively mitigating the risks of premature convergence and entrapment in local optima, thereby ensuring robust and reliable results. The convergence diagrams in Figs. 11, 12, 13, and 14 illustrate the algorithm's efficacy across LTS1, LTM2, LTL3, and LTV4 datasets. With a fixed

Table 10 Parameters configuration used in other algorithms

Algorithm	Hyperparameters	Values
Original HHO	Population Size (N)	5
	Upper search bound (UB)	1
	Lower search bound (LB)	0
	Number of iterations (T)	100
	Number of repeated runs (M)	20
	Fitness function constant	0.99
PSO	Inertia weight	0.7
	Momentum	0.9
	Social and cognitive learning factors	2.0
	Number of iterations (T)	100
GA	Population Size (N)	5
	Number of iterations (T)	100
	Crossover rate	0.8
	Mutation rate	0.1

iteration count of 100, the algorithm achieves impressive accuracies of 92.7% for LTS1, 91.1% for LTM2, 96.3% for LTL3, and 98.4% for LTV4.

The HHO algorithm exhibits resilience to problem dimensionality, delivering promising results with an intuitively minimal set of parameters. Tracking the average fitness function of the Hawks provides valuable insights into collective fitness dynamics throughout the optimization process. Additionally, the convergence criterion highlights fluctuations in the fitness value of the optimal solution (i.e., the rabbit) over the optimization lifecycle. The algorithm effectively navigates problem spaces with numerous variables by balancing exploratory and exploitative phases. These findings emphasize the HHO algorithm's versatility and robustness in addressing diverse problem classes, adeptly avoiding premature convergence and local optima. To analyze the impact of population size on optimization, we experimented with sizes of 5, 20, 50 and 100. Table 11 presents the results.

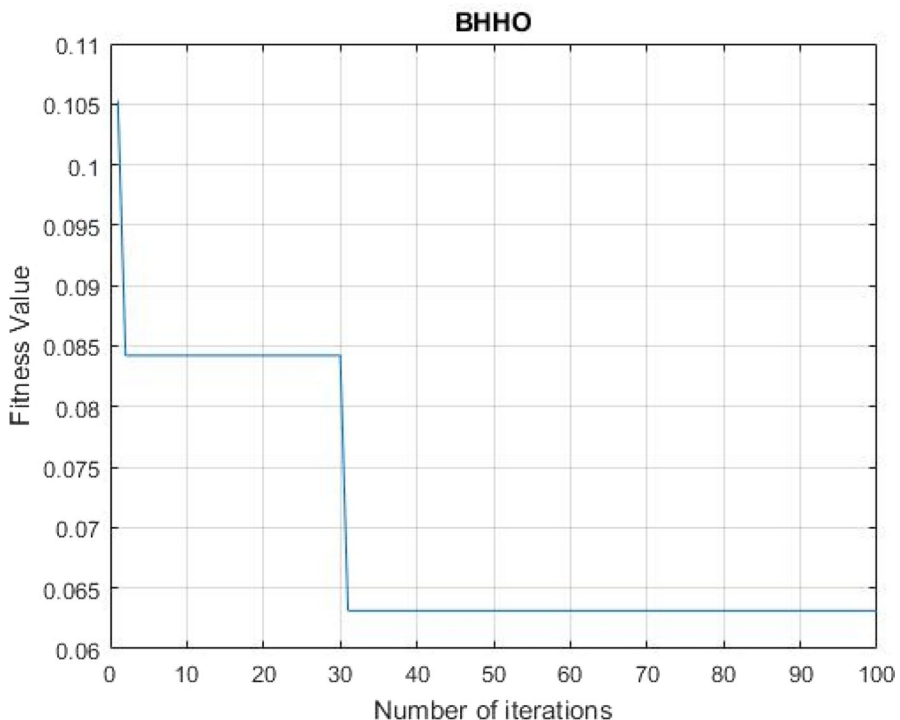


Fig. 11 Convergence plot with different iterations of the improved HHO on the 'LTS1' dataset

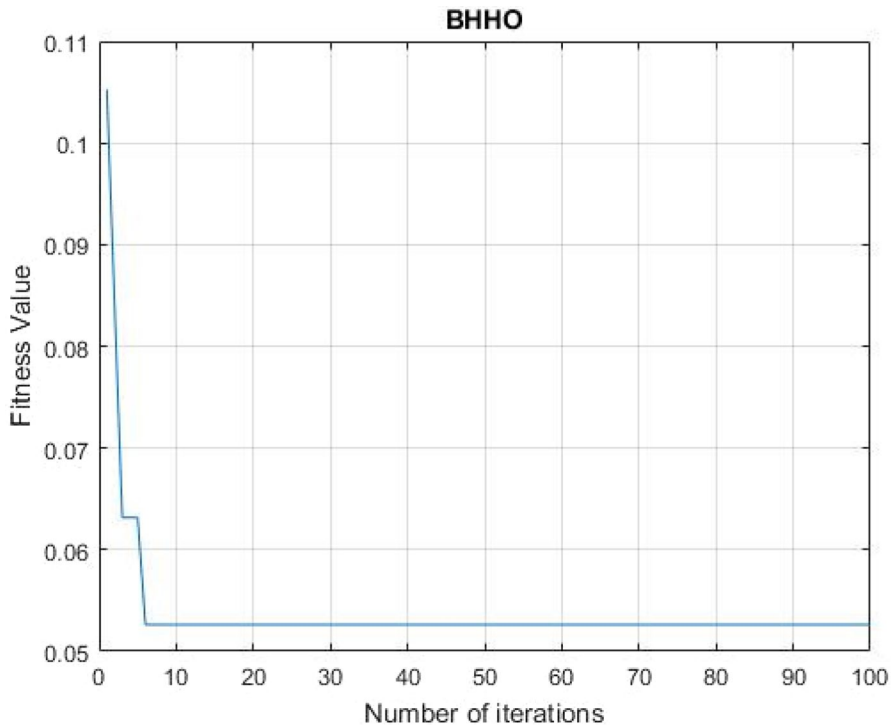


Fig. 12 Convergence plot with different iterations of the improved HHO on the 'LTM2' dataset

6 Discussion

This section compares the proposed approach's accuracy with various widely used algorithms and existing methods, as shown in Tables 12 and 13, utilizing the LVAT repository and the NASA dataset. The specifications of the datasets used for testing the proposed model are detailed in Sects. 4.1.1 and 4.1.2.

For comparison, Ref [42] employs the Crow Search Algorithm (CSA), Opposition-Based Learning (OBL), and Recurrent Neural Network (RNN) for feature selection in big data analysis. Reference [43] explores feature selection methods using Decision Tree (DT), RF, K-NN, and stacking algorithms. Reference [17] combines PSO and RF, while Ref. [44] integrates hybrid optimization methods with RF. Reference [45] combines the GWO algorithm with an SVM classifier (GWO+SVM) for anomaly detection. Similarly, Ref. [46] explores the Learnable Three-Line Hybrid Feature Fusion Algorithm (LTHFFA), which integrates kernel-based features (Binary Adaptive Variable Sparrow Search Algorithm (BAVSSA)), global features (Kernel Principal Component Analysis (KPCA)), and local features (Locally Linear Embedding (LLE)) for software defect prediction. Reference [29] investigates feature selection methods based on GOA, GA, and RF.

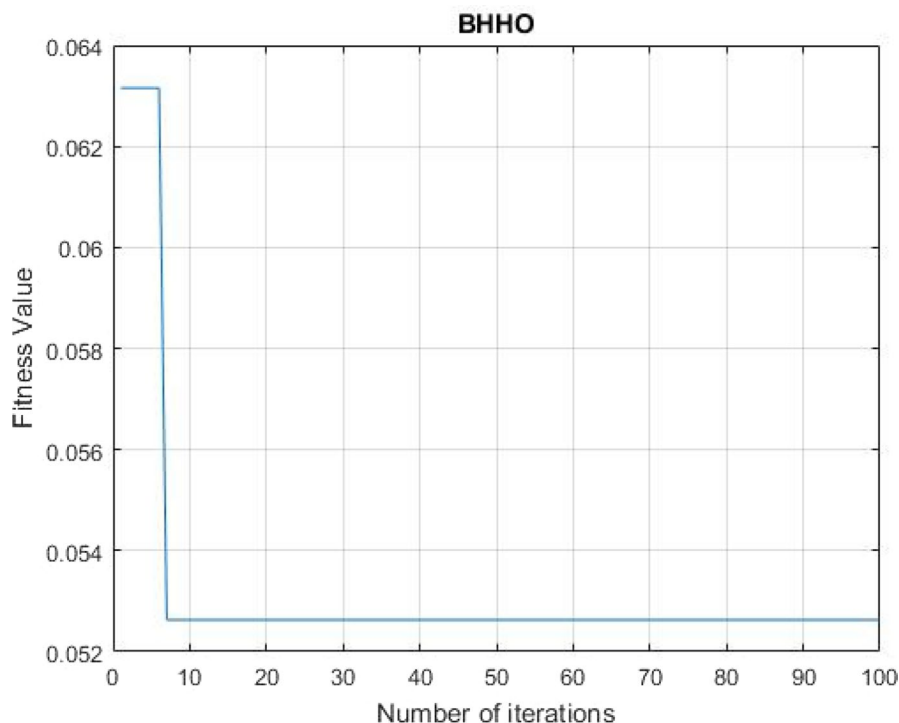


Fig. 13 Convergence plot with different iterations of the improved HHO on the 'LTL3' dataset

In contrast, this article proposes the BHHO algorithm combined with stacking-based ensemble learning. The comparative analysis provides valuable insights into the proposed method's efficacy and novelty relative to established approaches in the field.

These results demonstrate appropriate computational complexity, near-optimal accuracy, and strong performance compared to alternative techniques, highlighting the effectiveness of the proposed approach. High-dimensional datasets often introduce significant challenges in accurately identifying optimal features. Reducing the feature count enhances performance and decreases processing time, effectively addressing these challenges. Each technique, whether metaheuristic or machine learning, offers unique advantages and limitations, making integrating these methods crucial for balancing complexity, accuracy, and computational efficiency.

With their guided search mechanisms, Metaheuristic methods efficiently explore vast search spaces, making them particularly well-suited for complex problems like feature selection. Machine learning techniques, on the other hand, provide efficient tools for defect detection. The proposed hybrid approach combines the strengths of both paradigms. The integration's success is largely attributed to chaos theory within the HHO algorithm, which mitigates the risk of becoming trapped in local optima, alongside stacking-based ensemble learning, which enhances predictive accuracy and robustness.

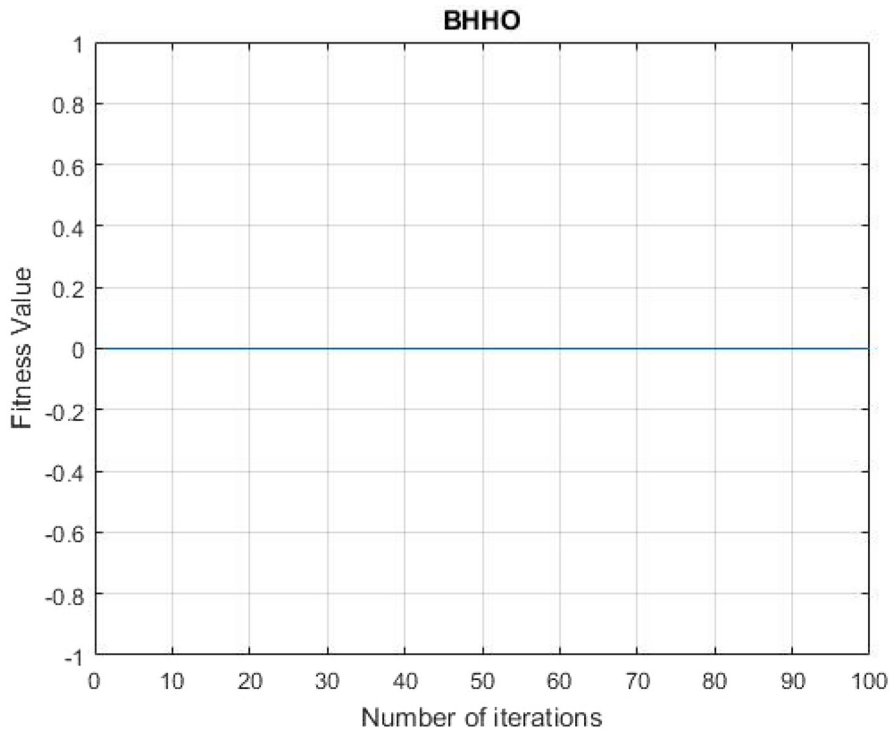


Fig. 14 Convergence plot with different iterations of the improved HHO on the 'LTV4' dataset

This synergy enables the hybrid approach to address feature selection challenges and defect detection effectively. In high-dimensional datasets, the proposed method outperforms single-algorithm approaches due to its enhanced global search capability, which allows it to navigate the search space comprehensively and avoid local optima. This makes the proposed approach particularly adept at handling the complexities of high-dimensional data, where traditional algorithms often encounter limitations.

However, the high performance of the proposed approach may be less significant under certain conditions. For instance, when the problem is simpler and does not require complex exploration or nonlinear behavior, HHO and COA provide simpler methods that may perform just as well with lower computational costs. Additionally, if the data is noisy, inconsistent, or lacks sufficient structure, the combined approach may overfit or fail to generalize effectively. Furthermore, the computational complexity of combining HHO, COA, and Ensemble Learning might lead to diminishing returns in problems with limited data or when the computational cost is too high relative to the problem's scale.

Table 11 Impact of Population Size on Optimization Accuracy and Computational Efficiency

Algorithm	Population size	Dataset	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)	Computational time (s)
COA + HHO	5	LVAT	95.3	92.6	94.8	92.7	1.3
	20		97.5	95.3	98.2	95.9	3.6
	50		97.8	95.7	98.6	97.3	7.3
	100		98.1	95.3	98.8	98.1	14.8
Original HHO	5		92.7	90.5	93.4	91.8	1.2
	20		96.3	94.2	97.1	95.6	3.5
	50		96.8	94.7	97.3	96.0	7.2
	100		97.2	95.1	97.6	96.4	14.5
PSO	5		91.5	89.2	92.0	90.6	1.5
	20		94.1	92.0	94.8	93.4	4
	50		95.2	93.5	95.9	94.7	8
	100		95.9	94.3	96.3	95.3	16
GA	5		90.3	88.0	91.2	89.6	1.8
	20		94.5	92.7	95.1	93.9	4.3
	50		95.8	94.0	96.0	95.0	9
	100		96.1	94.3	96.5	95.4	18
DE	5		93.1	91.0	94.0	92.5	1.4
	20		96.0	93.5	96.2	94.8	3.8
	50		96.7	94.5	97.0	95.7	7.5
	100		97.1	95.0	97.3	96.1	15

7 Conclusion and future work

Software Product Lines (SPLs) face numerous challenges, with selecting critical features and predicting software defects being among the most significant. This study proposes a novel hybrid approach that combines the Harris Hawks Optimization (HHO) metaheuristic algorithm with stack-based ensemble learning to address these issues. Integrating the COA within HHO mitigates local search pitfalls and enhances accuracy. The output from this phase feeds into the second stage of the ensemble learning algorithm, which further refines predictions. Experimental results on the LVAT and NASA datasets demonstrate that the proposed method achieves high accuracy and outperforms standard criteria compared to existing approaches. The proposed approach proves valuable for addressing optimization challenges and detecting defects in SPLs. Future work could explore integrating new metaheuristic algorithms with machine learning and deep learning techniques to improve the method further. Additionally, studies could focus on evaluating the approach with high-dimensional data. Furthermore, dynamic parameter tuning strategies may also be investigated to allow real-time adjustments during optimization, improving convergence rates and overall model

Table 12 Comparison of the final accuracy of the proposed method compared to previous methods on the LVAT dataset

Study (year)	Dataset	Method	Accuracy (%)
[17], 2024	LTS1	PSO + Random Forest	92.01
[18], 2024		GWO + SWM	90.00
[46], 2024		LTHFFA	91.65
Proposed method		HHO + Ensemble Stacking Classifier	92.7
[44], 2023	LTM2	Hybrid optimization technique combined + Random Forest	65.41
[46], 2024		LTHFFA	69.78
[18], 2024		GWO + SWM	67.22
Proposed method		HHO + Ensemble Stacking Classifier	91.1
[42], 2021	LTL3	CSA, OBL, RNN	95.19
[43], 2022		DT, RF, k-NN and stacking algorithm	95.07
[46], 2024		LTHFFA	94.28
Proposed method		HHO + Ensemble Stacking Classifier	96.3
[43], 2022	LTV4	DT, RF, k-NN and stacking algorithm	94.09
[29], 2024		Grasshopper Optimization Algorithm (GOA) + Genetic Algorithm (GA) + Random Forest (RF)	92.00
[17], 2024		PSO + Random Forest	91.57
Proposed method		HHO + Ensemble Stacking Classifier	98.4

Table 13 Comparison of the final accuracy of the proposed method compared to previous methods on the NASA dataset

Study (year)	Dataset	Method	Accuracy (%)
[42], 2021	CM1	CSA, OBL, RNN	94.74
[43], 2022		DT, RF, k-NN and stacking algorithm	95.23
[46], 2024		LTHFFA	96.11
Proposed method		HHO + Ensemble Stacking Classifier	97.91
[43], 2022	JM1	DT, RF, k-NN and stacking algorithm	96.81
[29], 2024		Grasshopper Optimization Algorithm (GOA) + Genetic Algorithm (GA) + Random Forest (RF)	94.49
[17], 2024		PSO + Random Forest	93.67
Proposed method		HHO + Ensemble Stacking Classifier	99.01
[17], 2024	KC1	PSO + Random Forest	92.89
[18], 2024		GWO + SWM	91.35
[46], 2024		LTHFFA	93.26
Proposed method		HHO + Ensemble Stacking Classifier	94.21
[44], 2023	PC1	Hybrid optimization technique combined + Random Forest	68.64
[46], 2024		LTHFFA	72.23
[18], 2024		GWO + SWM	69.51
Proposed method		HHO + Ensemble Stacking Classifier	90.93

accuracy. The source code for all algorithms presented in this paper is available for review and verification on GitHub (<https://github.com/Mehdi-Habibzadeh/EHHO-EL>).

Author contributions Author Contributions Author 1 (M.H.): Conceived the research idea, conducted data collection, and performed statistical analysis. Drafted the initial version of the manuscript and Writing review & editing. Author 2 (A.N.): Contributed to data analysis and interpretation and assisted in revising the initial manuscript draft. Author 3 (R.T.): Participated in designing the research methodology, contributed to the theoretical framework, and reviewed the final manuscript. Author 4 (H.R.): Assisted with resource collection, organized the results, and provided Project administration.

Data availability The authors have considered a public dataset, namely, LVAT and Nasa for experimental work: (<https://code.google.com/p/linux-variability-analysis-tools/source/browse/?repo=formulas>) (https://figshare.com/collections/NASA_MDP_Software_Defects_Data_Sets/4054940/1).

Declarations

Conflict of interest The authors declare no competing interests.

References

1. De Oliveira AL, Braga RTV, Masiero PC, Papadopoulos Y, Habli I, Kelly T, editors. Variability Management in Safety-Critical Software Product Line Engineering. New Opportunities for Software Reuse; 2018 2018//; Cham: Springer International Publishing.
2. Afzal U, Mahmood T, Usmani S (2022) Evolutionary computing to solve product inconsistencies in software product lines. *Sci Comput Program* 224:102875
3. Gao Y, Zhang J, Wang Y, Wang J, Qin L (2024) Love evolution algorithm: A stimulus–value–role theory-inspired evolutionary algorithm for global optimization. *J Supercomput* 80(9):12346–12407
4. Gao Y (2023) PID-based search algorithm: a novel metaheuristic algorithm based on PID algorithm. *Expert Syst Appl* 232:120886
5. Dehkordi M, Rastegari H, Nabiollahi-Najafabadi A, Javdani Gandomani T (2024) User Reviews Classification in Play Store Applications Using Deep Learning: An Empirical Study.
6. Zhao F, Zhuang C, Wang L, Dong C (2024) An Iterative Greedy Algorithm With \$Q\$-Learning Mechanism for the Multiobjective Distributed No-Idle Permutation Flowshop Scheduling. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*
7. Pan Z, Lei D, Wang L (2020) A knowledge-based two-population optimization algorithm for distributed energy-efficient parallel machines scheduling. *IEEE Trans Cybernetics* 52:5051–5063
8. Zhao F, Di S, Wang L (2022) A hyperheuristic with Q-learning for the multiobjective energy-efficient distributed blocking flow shop scheduling problem. *IEEE Transactions Cybernetics* 53:3337–3350
9. Markid HY, Dadaneh BZ, Moghaddam ME, editors. Bidirectional ant colony optimization for feature selection. 2015 The International Symposium on Artificial Intelligence and Signal Processing (AISP); 2015 3–5 March 2015.
10. Sayed GI, Darwish A, Hassanien AE (2018) A new chaotic whale optimization algorithm for features selection. *J Classif* 35:300–344
11. Hitesh CR, Kumari AC (2020) Feature selection optimization of healthcare software product line using BBO. *Proc Comput Sci* 167:1696–1704
12. Saber T, Brevet D, Botterweck G, Ventresque A (2021) Reparation in evolutionary algorithms for multi-objective feature selection in large software product lines. *SN Comput Sci* 2:138
13. Sun L, Yang Y, Liu Y, Ning T (2023) Feature selection based on a hybrid simplified particle swarm optimization algorithm with maximum separation and minimum redundancy. *Int J Mach Learn Cybern* 14:789–816

14. El-Kenawy E-SM, Mirjalili S, Alassery F, Zhang Y-D, Eid MM, El-Mashad SY, Aloyaydi BA, Ibrahim A, Abdelhamid AA (2022) Novel meta-heuristic algorithm for feature selection, unconstrained functions and engineering problems. *IEEE Access* 10:40536–40555
15. Liu G, Guo Z, Liu W, Jiang F, Fu E (2024) A feature selection method based on the golden Jackal-grey wolf hybrid optimization algorithm. *PLoS ONE* 19:e0295579
16. Potharlanka JL (2024) Feature importance feedback with Deep Q process in ensemble-based metaheuristic feature selection algorithms. *Sci Rep* 14:2923
17. Shanbhag A, Vincent S, Gowda SB, Kumar OP, Francis SAJ (2024) Leveraging metaheuristics for feature selection with machine learning classification for malicious packet detection in computer networks. *IEEE Access*
18. Nemati Z, Mohammadi A, Bayat A, Mirzaei A (2024) Metaheuristic and data mining algorithms-based feature selection approach for anomaly detection. *IETE J Re* 70(7):6040–6054
19. Azzeh M, Nassif AB, Talib MA, Iqba H (2024) Software defect prediction using non-dominated sorting genetic algorithm and k-nearest neighbour classifier. *Inform Softw Eng J* 18(1):240103
20. Gao Y, Wang J, Li C (2024) Escape after love: Philoponella prominens optimizer and its application to 3D path planning. *Clust Comput* 28:81
21. Maazalahi M, Hosseini S (2025) Machine learning and metaheuristic optimization algorithms for feature selection and botnet attack detection. *Know Inf Syst.* <https://doi.org/10.1007/s10115-024-02322-0>
22. Afzal MF, Khan I, Rashid J, Saddique M, Mohamed HG (2023) Binary oriented feature selection for valid product derivation in software product line. *Comput Mater Cont* 76:3653–3670
23. Tu S (2020) Optimisation-based training of evolutionary convolution neural network for visual classification applications. *IET Comput Vision* 14:259–267
24. Halim Z, Waqar M, Tahir M (2020) A machine learning-based investigation utilizing the in-text features for the identification of dominant emotion in an email. *Knowl-Based Syst* 208:106443
25. Jain S, Saha A (2021) Improving performance with hybrid feature selection and ensemble machine learning techniques for code smell detection. *Sci Comput Program* 212:102713
26. Suresh Kumar P, Behera HS, Nayak J, Naik B (2021) Bootstrap aggregation ensemble learning-based reliable approach for software defect prediction by using characterized code feature. *Innovations Syst Softw Eng* 17:355–379
27. Maâzoun J, Ben-Abdallah H, Bouassida N, editors. Clustering techniques for software product line feature identification. 2022 IEEE/ACS 19th International Conference on Computer Systems and Applications (AICCSA); 2022: IEEE.
28. Ali M, Mazhar T, Arif Y, Al-Otaibi S, Ghadi YY, Shahzad T, Khan MA, Hamam H (2024) Software defect prediction using an intelligent ensemble-based model. *IEEE Access*
29. Bakro M, Kumar RR, Husain M, Ashraf Z, Ali A, Yaqoob SI, Ahmed MN, Parveen N (2024) Building a Cloud-IDS by Hybrid Bio-Inspired Feature Selection Algorithms Along With Random Forest Model. *IEEE Access*
30. Mehta S, Patnaik KS (2021) Improved prediction of software defects using ensemble machine learning techniques. *Neural Comput Appl* 33:10551–10562
31. Büyükköçeci M, Okur MC (2022) A comprehensive review of feature selection and feature selection stability in machine learning. *Gazi Univ J Sci* 36:1506–1520
32. Al-Tashi Q, Kadir SJA, Rais HM, Mirjalili S, Alhussian H (2019) Binary optimization using hybrid grey wolf optimization for feature selection. *Ieee Access* 7:39496–39508
33. Uzma A-O, Tubaishat A, Shah B, Halim Z (2020) Gene encoder: a feature selection technique through unsupervised deep learning-based clustering for large gene expression data. *Neural Comput Appl* 34:1–23
34. Heidari AA, Mirjalili S, Faris H, Aljarah I, Mafarja M, Chen H (2019) Harris hawks optimization: algorithm and applications. *Futur Gener Comput Syst* 97:849–872
35. Faramarzi A, Heidarinejad M, Mirjalili S, Gandomi AH (2020) Marine predators algorithm: a nature-inspired metaheuristic. *Expert Syst Appl* 152:113377
36. Guo J, Liang JH, Shi K, Yang D, Zhang J, Czarnecki K, Ganesh V, Yu H (2019) SMTIBEA: a hybrid multi-objective optimization algorithm for configuring large constrained software product lines. *Softw Syst Model* 18:1447–1466
37. Singh LK, Khanna M, Garg H, Singh R (2023) Emperor penguin optimization algorithm- and bacterial foraging optimization algorithm-based novel feature selection approach for glaucoma classification from fundus images. *Soft Comput* 28:2431–2467

38. Arar ÖF, Ayan K (2015) Software defect prediction using cost-sensitive neural network. *Appl Soft Comput* 33:263–277
39. Czibula G, Marian Z, Czibula IG (2014) Software defect prediction using relational association rule mining. *Inf Sci* 264:260–278
40. Balaji G, Venkatesan R (2024) Optimized pyramidal convolution shuffle attention neural network based fetal cardiac cycle detection from echocardiograms. *Biomed Signal Process Control* 98:106701
41. Bakir H (2024) VoteDroid: a new ensemble voting classifier for malware detection based on fine-tuned deep learning models. *Multimed Tools Appl*. <https://doi.org/10.1007/s11042-024-19390-7>
42. SaiSindhuTheja R, Shyam GK (2021) An efficient metaheuristic algorithm based feature selection and recurrent neural network for DoS attack detection in cloud computing environment. *Appl Soft Comput* 100:106997
43. Kim Y-E, Kim Y-S, Kim H (2022) Effective feature selection methods to detect IoT DDoS attack in 5G core network. *Sensors* 22:3819
44. Bakro M, Kumar RR, Husain M, Ashraf Z, Ali A, Yaqoob SI, Ahmed MN, Parveen N (2024) Building a cloud-IDS by hybrid bio-inspired feature selection algorithms along with random forest model. *IEEE Access* 12:8846–8874
45. Nemati Z, Mohammadi A, Bayat A, Mirzaei A (2024) Metaheuristic and data mining algorithms-based feature selection approach for anomaly detection. *IETE J Res* 70:6040–6054
46. Tang Y, Dai Q, Du Y, Chen L, Niu X (2024) A software defect prediction method based on learnable three-line hybrid feature fusion. *Expert Syst Appl* 239:122409

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

**Mehdi Habibzadeh-khameneh^{1,2} · Akbar Nabiollahi-Najafabadi^{1,2} ·
Reza Tavoli³ · Hamid Rastegari^{1,2}**

✉ Akbar Nabiollahi-Najafabadi
a.nabi@pco.iaun.ac.ir

Mehdi Habibzadeh-khameneh
mehdi.habibzadeh@sco.iaun.ac.ir

Reza Tavoli
r.tavoli@iauc.ac.ir

Hamid Rastegari
rastegari@iaun.ac.ir

¹ Faculty of Computer Engineering, Najafabad Branch, Islamic Azad University, Najafabad, Iran

² Big Data Research Center, Najafabad Branch, Islamic Azad University, Najafabad, Iran

³ Department of Computer Engineering, Chalous Branch, Islamic Azad University, Chalous, Iran

Terms and Conditions

Springer Nature journal content, brought to you courtesy of Springer Nature Customer Service Center GmbH ("Springer Nature").

Springer Nature supports a reasonable amount of sharing of research papers by authors, subscribers and authorised users ("Users"), for small-scale personal, non-commercial use provided that all copyright, trade and service marks and other proprietary notices are maintained. By accessing, sharing, receiving or otherwise using the Springer Nature journal content you agree to these terms of use ("Terms"). For these purposes, Springer Nature considers academic use (by researchers and students) to be non-commercial.

These Terms are supplementary and will apply in addition to any applicable website terms and conditions, a relevant site licence or a personal subscription. These Terms will prevail over any conflict or ambiguity with regards to the relevant terms, a site licence or a personal subscription (to the extent of the conflict or ambiguity only). For Creative Commons-licensed articles, the terms of the Creative Commons license used will apply.

We collect and use personal data to provide access to the Springer Nature journal content. We may also use these personal data internally within ResearchGate and Springer Nature and as agreed share it, in an anonymised way, for purposes of tracking, analysis and reporting. We will not otherwise disclose your personal data outside the ResearchGate or the Springer Nature group of companies unless we have your permission as detailed in the Privacy Policy.

While Users may use the Springer Nature journal content for small scale, personal non-commercial use, it is important to note that Users may not:

1. use such content for the purpose of providing other users with access on a regular or large scale basis or as a means to circumvent access control;
2. use such content where to do so would be considered a criminal or statutory offence in any jurisdiction, or gives rise to civil liability, or is otherwise unlawful;
3. falsely or misleadingly imply or suggest endorsement, approval, sponsorship, or association unless explicitly agreed to by Springer Nature in writing;
4. use bots or other automated methods to access the content or redirect messages
5. override any security feature or exclusionary protocol; or
6. share the content in order to create substitute for Springer Nature products or services or a systematic database of Springer Nature journal content.

In line with the restriction against commercial use, Springer Nature does not permit the creation of a product or service that creates revenue, royalties, rent or income from our content or its inclusion as part of a paid for service or for other commercial gain. Springer Nature journal content cannot be used for inter-library loans and librarians may not upload Springer Nature journal content on a large scale into their, or any other, institutional repository.

These terms of use are reviewed regularly and may be amended at any time. Springer Nature is not obligated to publish any information or content on this website and may remove it or features or functionality at our sole discretion, at any time with or without notice. Springer Nature may revoke this licence to you at any time and remove access to any copies of the Springer Nature journal content which have been saved.

To the fullest extent permitted by law, Springer Nature makes no warranties, representations or guarantees to Users, either express or implied with respect to the Springer nature journal content and all parties disclaim and waive any implied warranties or warranties imposed by law, including merchantability or fitness for any particular purpose.

Please note that these rights do not automatically extend to content, data or other material published by Springer Nature that may be licensed from third parties.

If you would like to use or distribute our Springer Nature journal content to a wider audience or on a regular basis or in any other manner not expressly permitted by these Terms, please contact Springer Nature at

onlineservice@springernature.com